

ECE264-003 Fall 2019 Exam 1

8-9 PM, Thursday, Sep 19th, 2019

Name: _____ Login: _____

1. Pointer

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. int f1(int x, int y)
5. {
6.     int rtv;
7.     int * ptr = & y;
8.     * ptr = * ptr + 5;
9.     rtv = x - y;
10.    return rtv;
11. }
12.
13. int f2(int * x, int * y)
14. {
15.     int * z;
16.     int rtv;
17.     z = x;
18.     x = y;
19.     y = z;
20.     rtv = (* x) - (* y);
21.     return rtv;
22. }
23.
24. void f3(int * x, int * y)
25. {
26.     int z = 16;
27.     * y = * y + 2;
28.     x = &z;
29. }
30.
31. int main(int argc, char ** argv)
32. {
33.     int x = 15;
34.     int y = 3;
35.
36.     int outA = f1(x, y);
37.     // 1.A: what is the output?
38.     printf("Answer A, value of outA = %d\n", outA);
39.     // 1.B: what is the output?
40.     printf("Answer B, value of y = %d\n", y);
41.
42.     int outC = f2(&x, &y);
43.     // 1.C: what is the output?
44.     printf("Answer C, value of outC = %d\n", outC);
45.     // 1.D: what is the output?
46.     printf("Answer D, value of y = %d\n", y);
47.
48.     f3(&x, &y);
49.     // 1.E: what is the output?
50.     printf("Answer E, value of x = %d\n", x);
51.     // 1.F: what is the output?
52.     printf("Answer F, value of y = %d\n", y);
53.
54.     return EXIT_SUCCESS;
55. }
```

1.A: _____	1.B: _____	1.C: _____
1.D: _____	1.E: _____	1.F: _____

2. Call Stack

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4.
5. char f1(int x, int y)
6. {
7.     char c = x - y + 'a';
8.     return c;
9. }
10.
11. int main(int argc, char * * argv)
12. {
13.     if (argc < 3)
14.     {
15.         return EXIT_FAILURE;
16.     }
17.
18.     // 2.A: what is the output?
19.     printf("%ld\n", strlen(argv[1]));
20.
21.     char outB = * argv[1];
22.     // 2.B: what is the output?
23.     printf("%c\n", outB);
24.
25.     // 2.C: what is the output?
26.     char outC = f1(argv[1][1] - argv[2][0], argc);
27.     printf("%d\n", outC);
28.
29.     return EXIT_SUCCESS;
30. }
```

The program's name is q2prog. The program is called in a terminal using `./q2prog ECE 264 exam`

Questions 2.A - 2.C: please find the program output as the program finishes. Questions 2.D - 2.F: please fill in the following call stack as the program runs **after line 7, before line 8**. In the call stack, '-' means do not worry about it.

Frame	Symbol	Address	Value
f1	c	A202	2.D
	y	A201	-
	x	A200	-
	Value Address		2.E
	Return Location		2.F
main	outC	A103	-
	outB	A102	-
	argv	A101	-
	argc	A100	-

2.A: _____ 2.B: _____ 2.C: _____
2.D: _____ 2.E: _____ 2.F: _____

3. Makefile

Consider the following makefile:

```
1. WARNING = -Wall -Wshadow --pedantic
2. ERROR = -Wvla -Werror
3. GCC = gcc -std=c99 -g $(WARNING) $(ERROR)
4. TESTFALGS = -DTEST_FUNC
5.
6. SRCS = main.c compare.c testgen.c
7. # 3.A: fix the command to define OBJJS. Please include SRCS in the command.
8. # Specifically, you won't get points for answers like main.o compare.o testgen.o
9. OBJJS = [ 3.A ]
10.
11. q3prog: $(OBJJS)
```

```

12. $(GCC) $(TESTFALGS) $(OBSJ) -o q3prog
13.
14. # 3.B: fix the following command
15. .c.o:
16. $(GCC) $(TESTFALGS) [ 3.B ] $*.c
17.
18. testall: test1 test2
19.
20. test1: q3prog
21. ./q3prog inputs/input1 > output1
22. # 3.C: fix the command to compare (and ignore space) output1 and expected/expected1.
23. [ 3.C ] output1 expected/expected1
24.
25. test2: q3prog
26. ./q3prog inputs/input2 > output2
27. [ 3.C ] output2 expected/expected2
28.
29. clean: # remove all machine generated files
30. rm -f hw06 *.o output*

```

3.D: Suppose we define the following in the code.

```

# ifdef TEST_FUNC          # ifdef TEST_PRINT
void func1(void)           void func2(void)
{
    ...
}
# endif                    # endif

```

When we use the above makefile to test the program, which function is tested? A. func1; B. func2; C. Both func1 and func2; D. None of them.

3.E: Using this makefile, what is the Linux command to test the program with all test cases (both input1 and input2)?

3.F: Using this makefile, what is the Linux command to remove all machine generated files after testing?

3.A: _____	3.B: _____	3.C: _____
3.D: _____	3.E: _____	3.F: _____

4. Structure

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #pragma pack(1) // tell compiler not to pad any space
4. // assume sizeof(char) = 1, sizeof(int) = 4,
5. // sizeof(double) = 8, sizeof(a pointer) = 8
6.
7. #define NUMPOINT 5
8. typedef struct
9. {
10.     int x;
11.     int y;
12. } Point;
13.
14. int main(int argc, char ** argv)
15. {
16.     Point * arr;
17.     int ind;
18.
19.     // create an array of NUMPOINT Point objects
20.     // 4.A: fix the code
21.     arr = malloc( NUMPOINT * [ 4.A ] );
22.     if(arr == NULL)
23.     {
24.         return EXIT_FAILURE;
25.     }
26.
27.     // initialize the attributes
28.     for (ind = 0; ind < NUMPOINT; ind++)
29.     {
30.         arr[ind].x = ind;
31.         arr[ind].y = ind * ind + 5;
32.     }
33.
34.     Point * pptr = arr;
35.     // 4.B: what is the output?
36.     printf("Answer B = %d\n", pptr->x);
37.
38.     pptr += 2;
39.     // 4.C: what is the output?
40.     printf("Answer C = %d\n", pptr->y);
41.
42.     int * aptr = & arr[1].y;
43.     // 4.D: what is the output?
44.     printf("Answer D = %d\n", * aptr);
45.
46.     aptr += 3;
47.     // 4.E: what is the output?
48.     printf("Answer E = %d\n", * aptr);
49.
50.     // 4.F: If the memory is not released, how many
51.     // bytes of memory are leaked?
52.     free(arr);
53.
54.     return EXIT_SUCCESS;
55. }
```

4.A: _____	4.B: _____	4.C: _____
4.D: _____	4.E: _____	4.F: _____

ASCII Table

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL	32	SPACE	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	'	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL